

CLOSECUT LIBRARY / VOLUME 04

Backend, Infrastructure & Security

The Public Platform Architecture Edition

PUBLIC READING EDITION

Version 1.0 · July 2026

Contents

- 01 Platform Foundation
- 02 System Context and Boundaries
- 03 Identity and Account Lifecycle
- 04 Authorization and Private Sharing
- 05 Data Ownership and Lifecycle
- 06 Local-First Synchronization
- 07 Conflict Resolution and Idempotency
- 08 Security and Privacy Principles
- 09 Privacy, Compliance, and User Rights
- 10 Reliability and Observability
- 11 Cross-Platform Backend Contract
- 12 Future Platform Evolution
- 13 Platform Governance
- 14 Public Platform Decision Records
- Appendix / Public architecture diagrams

Platform Foundation

CloseCut uses a local-first mobile architecture with Firebase-backed identity and synchronization. The public platform model is intentionally described at the level of responsibilities and boundaries rather than deployable configuration.

Principles

- Private data remains private unless the user explicitly shares it.
- Local state provides immediate responsiveness and offline continuity.
- Remote services replicate supported data and enforce shared-access boundaries.
- Server-side policy, not UI visibility, is the authorization boundary.
- Operational claims are labeled as current, near-term, or future.

Public boundary

This edition does not publish production topology, deployment instructions, environment identifiers, secrets, rule source, index definitions, runbooks, or security-audit evidence.

System Context and Boundaries

The platform has four conceptual layers: the iOS client, local persistence, synchronization and repository services, and Firebase-managed remote services.

The client owns presentation and user intent. SwiftData preserves immediate local state. Repository and synchronization layers translate between domain state and remote representations. Firebase Authentication establishes identity, while Firestore, Storage, and privileged backend operations enforce remote access and shared-state invariants.

Boundary rules

- Views do not directly own remote authorization logic.
- Remote outages should not erase valid local work.
- Shared access is scoped to explicit membership and explicit sharing.
- Privileged cross-document operations belong behind trusted backend boundaries.
- Platform-specific implementation must preserve the same domain contract across iOS and Android.

Identity and Account Lifecycle

Authentication establishes a principal for remote operations, but identity alone does not grant access to all data.

The public account model covers sign-in, profile creation, session transitions, recovery, sign-out, and account deletion. Each transition must preserve privacy and avoid leaking state between users on the same device.

Requirements

- Authentication state changes must be explicit and observable.
- User-owned data remains scoped to the authenticated principal.
- Session loss must not silently discard unsynchronized local work.
- Account deletion requires coordinated local and remote cleanup.
- Recovery and deletion flows must avoid exposing whether another person's account exists.

Exact provider configuration, production credentials, and deletion runbooks are excluded from this edition.

Authorization and Private Sharing

CloseCut is personal first and social second. Entries begin private. Sharing requires an explicit action and an explicitly selected Circle.

Circle access is membership-based. Ownership and membership are distinct concepts: owners manage the Circle, while active members receive only the access permitted by policy. Client checks may improve usability, but remote policy must enforce every protected read and write.

Public access model

- User-owned private data: accessible only to its owner.
- Circle metadata: accessible only to permitted members.
- Shared entries: accessible only when both the share relationship and membership are valid.
- Reactions and comments: scoped to content the participant is allowed to see.
- Invite flows: validated through trusted backend operations rather than public enumeration.

Exact rule expressions and collection paths are intentionally omitted.

Data Ownership and Lifecycle

The data model separates personal history, shared context, and operational metadata. Ownership is explicit and stable across synchronization.

Lifecycle principles

- Store only what the product needs.
- Use stable identifiers for logical entities and retry-safe operations.
- Represent deletion with tombstones long enough to synchronize safely.
- Define retention and permanent deletion separately.
- Evolve schemas compatibly or through staged migration.
- Keep analytics and diagnostics free of private journal content.

This edition describes the policy but omits exact collection names, field dictionaries, document paths, and migration scripts.

Local-First Synchronization

CloseCut commits supported user actions locally before attempting remote replication. This protects responsiveness and preserves work during network or service failure.

A durable pending-action model represents remote work that has not yet completed. Actions move through explicit states, retain retry information, and must be safe to execute more than once.

Sync contract

1. Validate user intent locally.
2. Commit the local domain change.
3. Enqueue the corresponding remote action.
4. Attempt synchronization when conditions allow.
5. Reconcile confirmed remote state.
6. Preserve actionable failure information without exposing sensitive content.

The public edition excludes internal queue schemas, operational telemetry fields, and recovery runbooks.

Conflict Resolution and Idempotency

Distributed state can diverge when devices operate offline, retries overlap, or membership changes during an active session. CloseCut therefore requires deterministic reconciliation.

Rules

- Repeating an operation must not create duplicate logical effects.
- Stable operation and entity identifiers support retry safety.
- Deletion tombstones prevent removed data from being resurrected.
- Explicit sharing changes are treated as deliberate state transitions.
- Membership changes invalidate access even when old content remains visible locally.
- Old clients must not silently overwrite newer schema semantics.

Conflict policy should favor preserving user-authored local work while maintaining remote privacy constraints.

Security and Privacy Principles

CloseCut's security posture begins with data minimization, explicit authorization, safe logging, and conservative sharing.

Security principles

- Authentication identifies; authorization decides.
- Backend policy is authoritative.
- Private content is never used as analytics payload.
- Inputs are validated at every trust boundary.
- Secrets and credentials stay outside source and public documentation.
- Privileged operations execute in trusted backend contexts.
- Security controls are tested with both permitted and denied cases.

Privacy principles

- Private by default.
- Explicit sharing only.
- Purpose-limited collection.
- Clear account and deletion controls.
- No advertising model dependent on intimate personal content.

Detailed threat scenarios, abuse controls, enforcement configuration, and incident procedures are excluded.

Privacy, Compliance, and User Rights

Public-facing privacy behavior must align with the app's actual implementation and store disclosures.

The platform must support understandable consent, accurate privacy labels, data minimization, deletion, retention, and appropriate handling of third-party services. TMDB attribution and Apple platform requirements are treated as product obligations, not optional marketing details.

Public commitments

- Explain what data is used and why.
- Avoid collecting content that is not required.
- Keep analytics separate from journal text and social content.
- Provide account deletion and support paths.
- Review regional and age-related requirements before expanding scope.
- Keep policy, product behavior, and App Store disclosures consistent.

This edition does not make unsupported claims of legal compliance.

Reliability and Observability

Reliability means preserving trust, not merely maximizing uptime. The local-first model allows the app to remain useful during remote degradation, while observability helps detect failures in synchronization and backend services.

Public operational model

- Monitor service health and synchronization outcomes without capturing private content.
- Use structured, privacy-safe logs.
- Detect elevated failures and crash patterns.
- Bound remote usage and cost.
- Maintain recovery and rollback strategies for material platform changes.
- Treat backup, recovery, and incident response as planned capabilities rather than assumptions.

Alert thresholds, dashboards, billing controls, environment topology, and runbooks remain private.

Cross-Platform Backend Contract

iOS and Android may use different local technologies, but they must preserve the same remote contract and product semantics.

Shared requirements

- Equivalent ownership and visibility rules.
- Equivalent authentication and account lifecycle behavior.
- Equivalent synchronization semantics.
- Compatible identifiers, timestamps, and deletion markers.
- Versioned evolution that supports older clients during migration.
- Platform-specific UI differences must not weaken privacy or authorization.

The backend contract should remain portable enough to support future platform or infrastructure changes without rewriting the product model.

Future Platform Evolution

Future capabilities may include group decision support, taste modeling, cinema intelligence, shared memories, notifications, advanced search, and larger-scale recommendations.

These capabilities must remain optional, privacy-reviewed, cost-bounded, and clearly separated from current implementation.

Evolution rules

- Add new data only when a product requirement justifies it.
- Prefer derived insights over unnecessary raw-data replication.
- Avoid public social defaults.
- Preserve local-first behavior where it supports trust.
- Keep migration paths open beyond Firebase-specific implementation.
- Do not describe future infrastructure as deployed.

Platform Governance

Platform changes require explicit ownership, review, evidence, and a deprecation path.

Governance model

- Schema changes receive compatibility review.
- Security-sensitive changes receive security review.
- Data-use changes receive privacy review.
- Operational changes identify an owner and rollback path.
- Documentation is updated with behavior-changing releases.
- Deprecated contracts remain supported for a defined transition period.
- Decision records capture durable architectural choices.

Internal release gates, ownership assignments, and incident escalation details are not part of the public edition.

Public Platform Decision Records

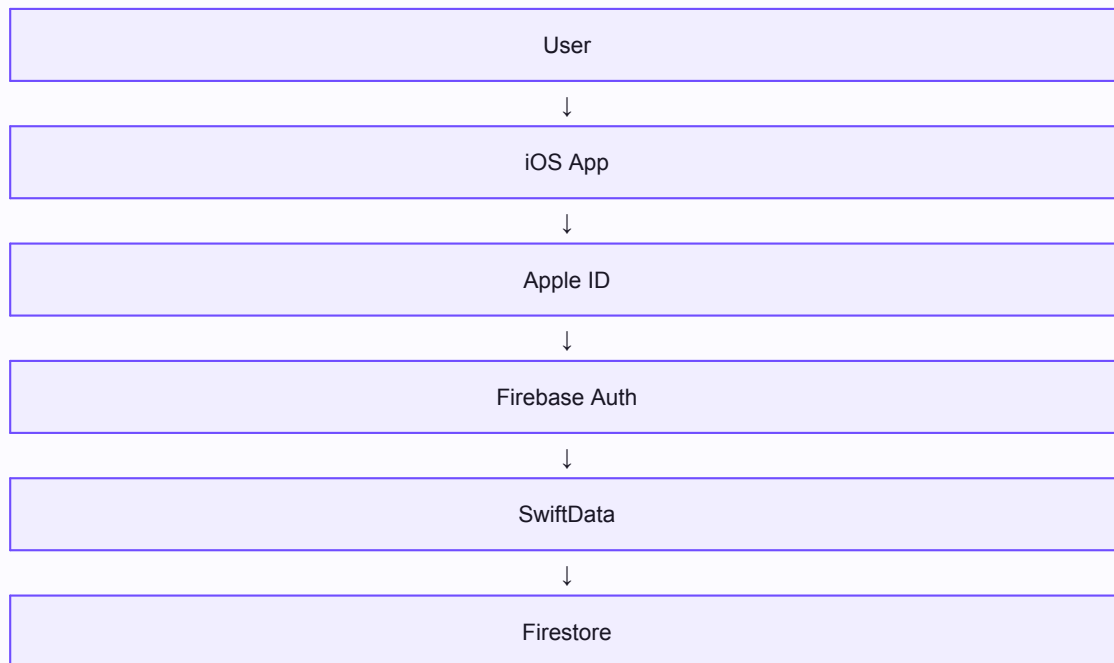
This chapter summarizes the stable public platform decisions included with this edition.

- SwiftData is the immediate UI source of truth.
- Firestore acts as a remote synchronization and trusted-sharing layer.
- Supported user writes commit locally first.
- Sharing is always explicit.
- Circle access depends on active membership.
- Retryable remote writes are idempotent.
- Pending actions represent durable remote work.
- Backend policy enforces protected access.
- Privileged cross-document operations belong in trusted backend services.
- Data collection is minimized.
- Private content is excluded from analytics.

The detailed records are included in [decision-records/](#).

Authentication Flow

Public architecture diagram

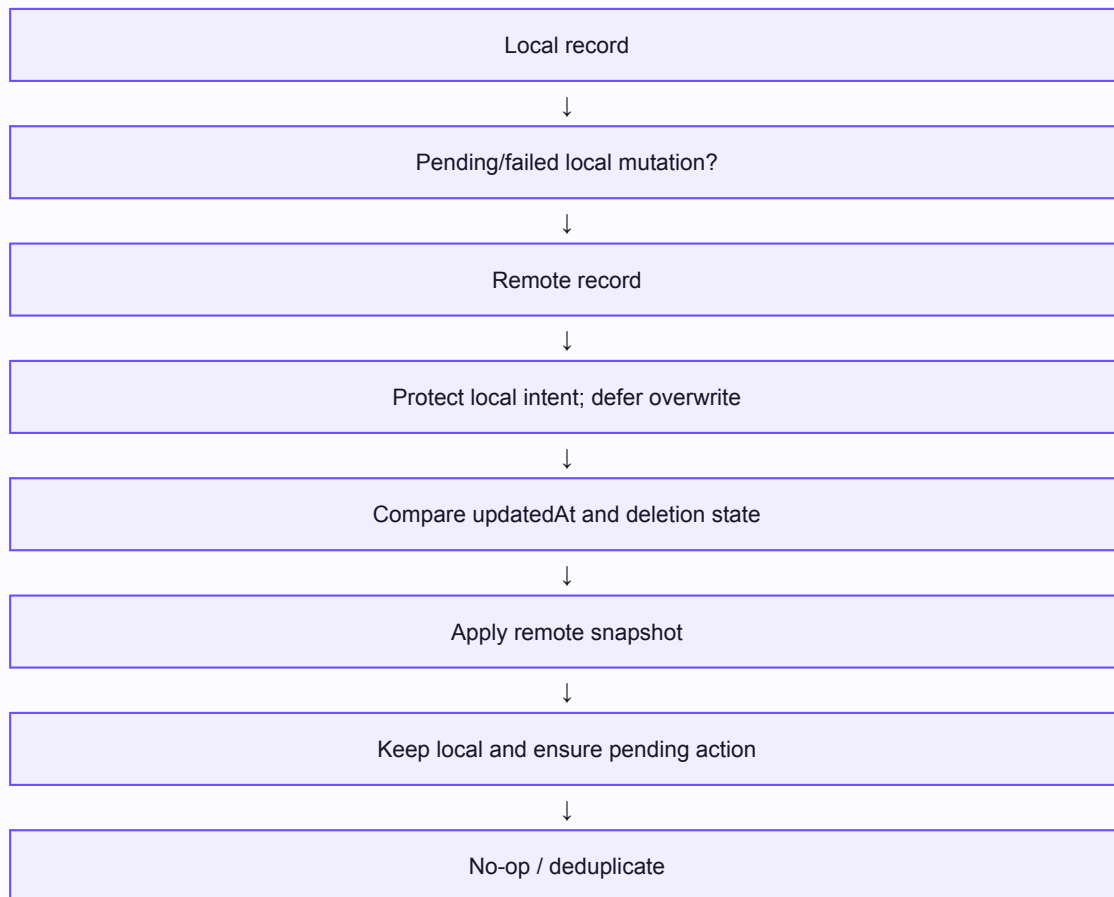


Key interactions

- A: Choose Sign in with Apple
- Apple: Authorization request + nonce
- A: Identity token
- Auth: OAuth credential
- A: Firebase UID/session
- Local: Resolve user-scoped local state
- FS: Create/merge /users/{uid}
- A: Profile state or rule error

Conflict Resolution

Public architecture diagram

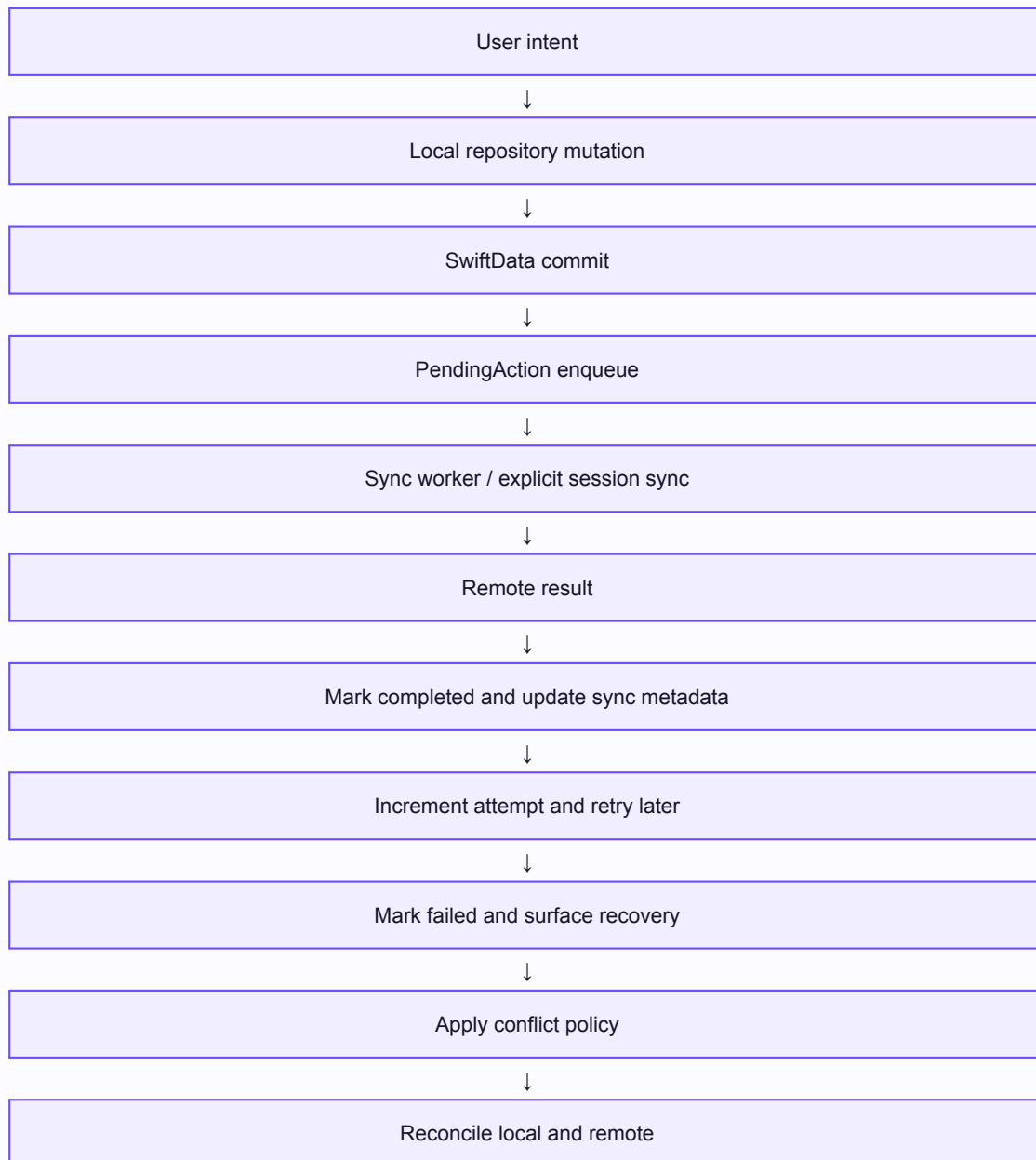


Key interactions

- yes| L[Protect local intent; defer overwrite]
- no| T{Compare updatedAt and deletion state}
- remote newer| R[Apply remote snapshot]
- local newer| Q[Keep local and ensure pending action]
- same logical state| N[No-op / deduplicate]

Local First Sync

Public architecture diagram

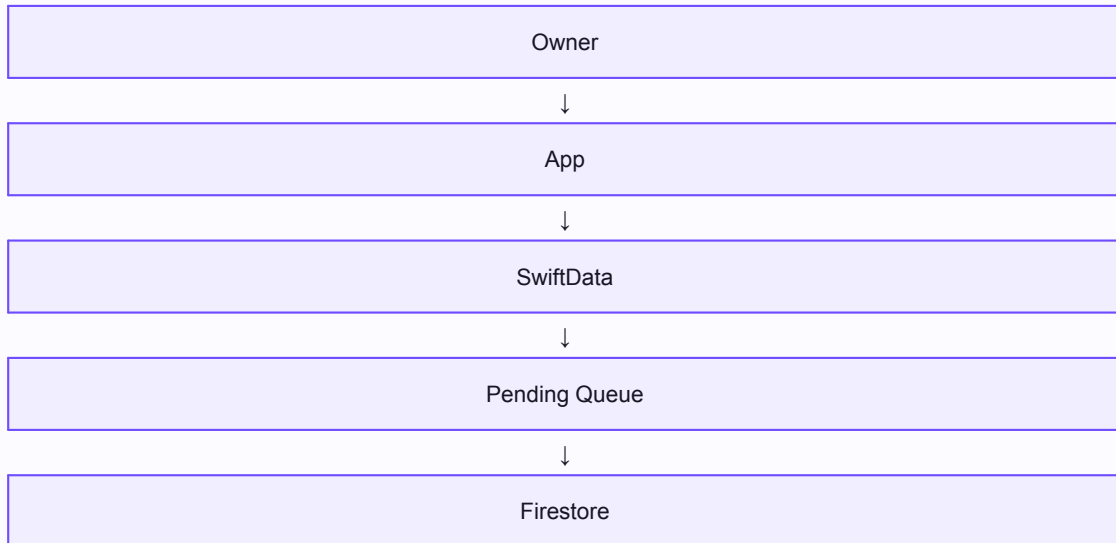


Key interactions

- success| C[Mark completed and update sync metadata]
- transient| B[Increment attempt and retry later]
- permanent| F[Mark failed and surface recovery]
- conflict| P[Apply conflict policy]

Share Entry Flow

Public architecture diagram



Key interactions

- A: Share entry to selected Circle(s)
- L: Set visibility=circle and sharedCircleIds
- Q: Enqueue updateVisibility/updateEntry
- FS: Idempotent owner-authorized update
- Q: Rule-validated result

System Context

Public architecture diagram

